



**UNIVERSITÀ
DI TORINO**

UNIVERSITÀ DEGLI STUDI DI TORINO

Corso di Laurea Magistrale in Fisica dei Sistemi Complessi

Improving VAE Representational Capabilities for Understanding LLMs

Tesi di Laurea Magistrale

Relatore:

Prof. Michele Caselle

Corelatori:

Prof. Alan Perotti

Prof. André Panisson

Controrelatore:

Prof. Matteo Osella

Candidato:

Francesco Marchisotti

Tutor Aziendale:

Gabriele Ciravegna

Anno Accademico 2025/2026

Francesco Marchisotti: *Improving VAE Representational Capabilities for Understanding LLMs*, Fisica dei Sistemi Complessi

CENTAI TEAM:

Gabriele Ciravegna

Alan Perotti

André Panisson

Francesco Cozzi

UNITO PROFESSORS:

Michele Caselle

Matteo Osella

LOCATION:

Torino

TIME FRAME:

Oct. 2025 - Jul. 2026

“I wish there was a way to know you’re in the good old days
before you’ve actually left them.”

— *Andy Bernard*

ABSTRACT

Short summary of the contents in English... a great guide by Kent Beck how to write good abstracts can be found here:

<https://plg.uwaterloo.ca/~migod/research/beck00PSLA.html>

ZUSAMMENFASSUNG

Kurze Zusammenfassung des Inhaltes in deutscher Sprache...

ACKNOWLEDGMENTS

To whom shall be acknowledged,
you are being acknowledged with this exact statement.

“I’m so funny.”

— The candidate, who’s not actually funny

CONTENTS

1	INTRODUCTION	1
1.1	Motivation	1
1.2	Thesis Structure	1
2	BACKGROUND AND RELATED WORK	3
2.1	NLP	3
2.2	XAI	3
2.2.1	Traditional XAI	3
2.2.2	C-XAI	3
2.3	AutoEncoders	3
2.3.1	Standard AutoEncoders	3
2.3.2	Variational AutoEncoders	5
2.3.3	Sparse AutoEncoders	9
3	METHODS	15
3.1	Definitions/Notation	15
3.2	Aim of the Work	15
3.3	Model Development	15
3.3.1	Probabilistic Formulation	15
3.3.2	Architectures	15
4	EXPERIMENTS	17
4.1	Model Training	17
4.2	Evaluation Metrics	17
4.2.1	MSE	17
4.2.2	ℓ_0	17
4.2.3	Perplexity	17
4.3	Results	17
5	CONCLUSIONS	19
5.1	Contributions	19
5.2	Future Work	19
	BIBLIOGRAPHY	21

LIST OF FIGURES

Figure 2.1	Illustration of Denoising AutoEncoder model architecture, reproduced from Weng [32]. <i>Note: this figure uses f for the decoder and g for the encoder, the reverse of the letter convention adopted in this thesis.</i>	5
Figure 2.2	Two-dimensional latent spaces learned on MNIST, coloured by digit class, reproduced from Shafkat [27]. Optimising for reconstruction alone (left) lets a standard AutoEncoder separate the classes into disconnected clusters with gaps that decode to unrealistic samples, whereas the Variational AutoEncoder objective (right) yields a smoother, more continuous arrangement, concentrated around the origin by the prior, that can be sampled and interpolated.	6
Figure 2.3	Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs [19].	10

LIST OF TABLES

LISTINGS

ACRONYMS

AE	AutoEncoder	3, 5, 8 f., 12
ELBO	Evidence Lower Bound	6 ff.

KL	Kullback–Leibler	7f.
LLM	Large Language Model	12
MECHINT	Mechanistic Interpretability	9, 12
MLP	Multi-Layer Perceptron	12f.
MSE	Mean Squared Error	3, 7
PCA	Principal Component Analysis	4
SAE	Sparse AutoEncoder	9–13
VAE	Variational AutoEncoder	5–9, 12

INTRODUCTION

1.1 MOTIVATION

1.2 THESIS STRUCTURE

BACKGROUND AND RELATED WORK

2.1 NLP

2.2 XAI

2.2.1 *Traditional XAI*

2.2.2 C-XAI

2.2.2.1 *Explainable-by-design Models*

CBM/CEM

LF-CBM

2.2.2.2 *Post-hoc Methods*

T-CAV

2.3 AUTOENCODERS

An AutoEncoder (AE) is a neural network trained to reconstruct its input through an intermediate latent representation [11, Ch 14]. It consists of two components: an **encoder** $f_\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m$ that maps an input $\mathbf{x}$ to a latent code $\mathbf{z} = f_\phi(\mathbf{x})$, and a **decoder** $g_\theta : \mathbb{R}^m \rightarrow \mathbb{R}^n$ that reconstructs the input as $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$. Training minimises a reconstruction loss, typically the Mean Squared Error (MSE):

$$\mathcal{L}_{\text{recon}} = \text{MSE}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2. \quad (2.1)$$

When $m < n$, the bottleneck forces the model to learn a compressed representation. When $m > n$, the network is overcomplete and additional constraints are required to prevent the trivial solution of learning the identity. In either case, the structure of the latent space is shaped by the choice of architecture and regularisation.

2.3.1 *Standard AutoEncoders*

2.3.1.1 *Applications*

IMAGE RETRIEVAL Krizhevsky and Hinton [17] proposed using deep autoencoders for content-based image retrieval. Their key insight is that a deep autoencoder can learn to map images to short binary

codes that preserve semantic similarity: images that are perceptually similar are mapped to nearby codes in the latent space, allowing retrieval to be reduced to a fast lookup rather than an exhaustive search over the full database. They demonstrate this on a dataset of 1.6 million tiny images, showing that codes produced by deep autoencoders yield qualitatively and quantitatively better retrieval results than matching in raw pixel space. This work illustrated that the compressed latent representation learned by an autoencoder need not merely serve reconstruction, but can itself be a semantically meaningful and practically useful object.

ANOMALY DETECTION Sakurada and Yairi [26] were among the first to explicitly propose autoencoders as a tool for anomaly detection. Their core insight is that an autoencoder trained on normal data learns a compressed representation of the normal data distribution; at test time, inputs that deviate from this distribution are reconstructed poorly, and the resulting reconstruction error serves as an anomaly score. To substantiate this claim, they apply autoencoders to both synthetic data generated from the Lorenz system and real spacecraft telemetry, comparing against linear Principal Component Analysis (PCA) and kernel PCA. They demonstrate that autoencoders, by virtue of their nonlinear dimensionality reduction, are able to detect subtle anomalies that linear PCA misses, while remaining computationally lighter than kernel PCA. This work established reconstruction error as a natural unsupervised anomaly signal for autoencoder-based methods.

2.3.1.2 Denoising AutoEncoders

The denoising autoencoder was introduced by Vincent et al. [31] to avoid overfitting and improve the robustness of the learned representation. The input is partially corrupted by adding noise to or masking some values of the input vector in a stochastic manner, $\mathbf{x}_c \sim \mathcal{M}_{\mathcal{D}}(\mathbf{x}_c \mid \mathbf{x})$, as shown in fig. 2.1. The model is then trained to reconstruct the original uncorrupted input:

$$\mathcal{L}_{\text{DAE}} = \|\mathbf{x} - g_{\theta}(f_{\phi}(\mathbf{x}_c))\|_2^2. \quad (2.2)$$

This design is motivated by the fact that humans can easily recognize an object or a scene even the view is partially occluded or corrupted. To “repair” the partially destroyed input, the denoising autoencoder has to discover and capture relationship between dimensions of input in order to infer missing pieces.

For high dimensional input with high redundancy, like images, the model is likely to depend on evidence gathered from a combination of many input dimensions to recover the denoised version rather than to overfit one dimension. This builds up a good foundation for learning *robust* latent representation.

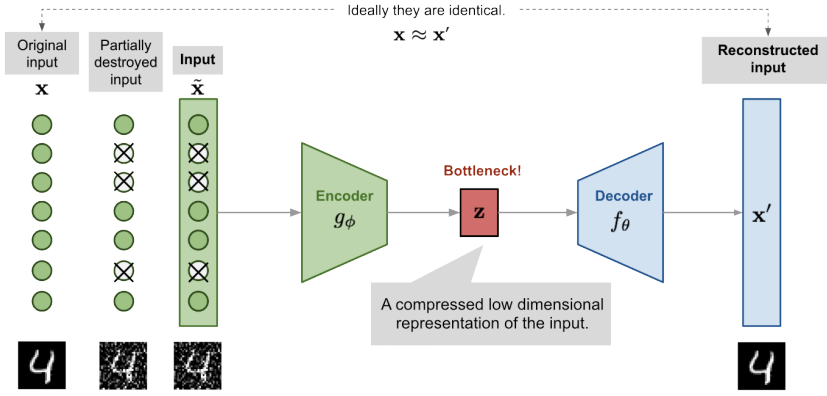


Figure 2.1: Illustration of Denoising AutoEncoder model architecture, reproduced from Weng [32]. Note: this figure uses f for the decoder and g for the encoder, the reverse of the letter convention adopted in this thesis.

The noise is controlled by a stochastic mapping $\mathcal{M}_{\mathcal{D}}(\mathbf{x}_c | \mathbf{x})$, and it is not specific to a particular type of corruption process (i. e., masking noise, Gaussian noise, salt-and-pepper noise, etc.). Naturally the corruption process can be equipped with prior knowledge [32].

2.3.2 Variational AutoEncoders

Variational AutoEncoders (VAEs) [15, 24] reinterpret the autoencoder as a probabilistic generative model. Rather than mapping an input to a single latent point, the encoder maps it to a *distribution* over latent codes, and the decoder is treated as a generative model that defines a distribution over inputs given a latent code. This probabilistic framing turns the autoencoder into a model from which new data can be sampled, and forces its latent space to have a smooth, continuous structure.

This smoothness is the property that sets a VAE apart from a standard AE. Trained for reconstruction alone, an AE is free to scatter its encodings into well-separated clusters with large gaps between them; those gaps are regions that decode to unrealistic outputs, so the space cannot be sampled or interpolated reliably. The VAE objective instead pushes encodings to fill the latent space more uniformly and to vary smoothly, so that nearby codes decode to similar inputs and paths between codes stay on the data manifold. Figure 2.2 illustrates the difference on a two-dimensional latent space trained on MNIST.

2.3.2.1 Variational Inference

Formally, a VAE models the data through a generative process in which a latent code is first drawn from a prior $\mathbf{z} \sim p(\mathbf{z})$ — typically a standard Gaussian $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ — and an observation is then

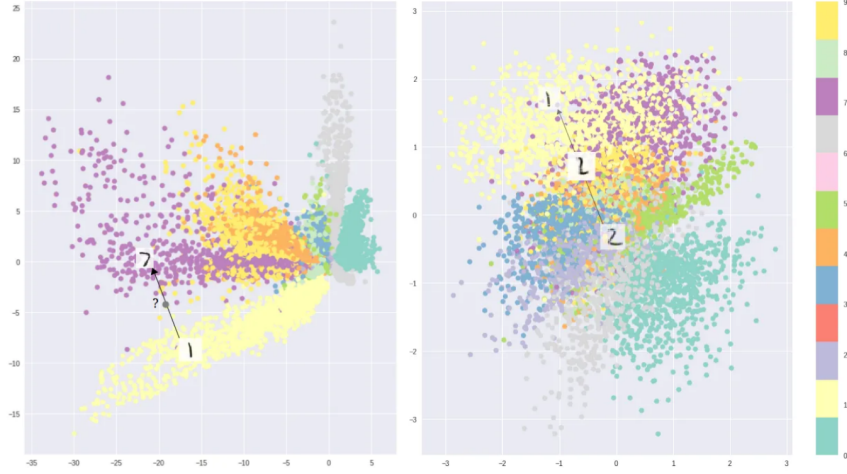


Figure 2.2: Two-dimensional latent spaces learned on MNIST, coloured by digit class, reproduced from Shafkat [27]. Optimising for reconstruction alone (left) lets a standard AutoEncoder separate the classes into disconnected clusters with gaps that decode to unrealistic samples, whereas the Variational AutoEncoder objective (right) yields a smoother, more continuous arrangement, concentrated around the origin by the prior, that can be sampled and interpolated.

drawn from a decoder distribution $p_\theta(\mathbf{x} \mid \mathbf{z})$ parameterised by a neural network. Training would ideally maximise the marginal likelihood

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z}) d\mathbf{z},$$

but this integral, and the corresponding posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$, are intractable for any non-trivial decoder.

The key idea of Kingma and Welling [15] is to sidestep this intractability with *amortised variational inference*: an encoder network $q_\phi(\mathbf{z} \mid \mathbf{x})$ is trained to approximate the true posterior, with all data points sharing the same inference parameters ϕ . The encoder outputs the parameters of a diagonal Gaussian,

$$q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag}(\sigma_\phi^2(\mathbf{x}))), \quad (2.3)$$

so that each input is mapped to a mean and a (diagonal) covariance rather than to a single point.

Instead of the intractable likelihood, the VAE maximises a tractable lower bound on it, the Evidence Lower Bound (ELBO):

$$\log p_\theta(\mathbf{x}) \geq \underbrace{\mathbb{E}_{q_\phi(\mathbf{z} \mid \mathbf{x})} [\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \underbrace{D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))}_{\text{regularisation}}. \quad (2.4)$$

The bound follows from the identity

$$\log p_\theta(\mathbf{x}) = \text{ELBO} + D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p_\theta(\mathbf{z} \mid \mathbf{x})),$$

together with the non-negativity of the Kullback–Leibler (KL) divergence; it is tight precisely when the approximate posterior matches the true posterior $p_\theta(\mathbf{z} \mid \mathbf{x})$. Maximising the ELBO simultaneously pushes the decoder to assign high likelihood to the data (the reconstruction term) and keeps the approximate posterior close to the prior (the KL divergence term). Negating it yields the training loss:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})). \quad (2.5)$$

When the decoder is Gaussian with fixed isotropic variance, the first term reduces, up to an additive constant, to the MSE reconstruction loss of eq. (2.1); the second term, being the divergence between two Gaussians, has a closed form. For the standard-normal prior it becomes:

$$D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) = \frac{1}{2} \sum_{j=1}^m (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1),$$

where μ_j and σ_j are the components of the encoder’s predicted mean and standard deviation.

Evaluating the reconstruction term requires sampling $\mathbf{z}$ from $q_\phi(\mathbf{z} \mid \mathbf{x})$, an operation that is not differentiable with respect to the encoder parameters and would therefore block gradient-based training. The VAE resolves this with the *reparameterisation trick*: the stochasticity is moved to an auxiliary noise variable, and the sample is expressed as a deterministic, differentiable function of the encoder outputs,

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\odot$ denotes the element-wise product. Gradients can now flow through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$ while the randomness is confined to $\boldsymbol{\epsilon}$, making the whole objective trainable end-to-end by stochastic gradient descent.

2.3.2.2 Architectures

There exist several variants that modify the base VAE to shape the latent space or the form of the latent code; two are particularly relevant here.

β -VAE Higgins et al. [13] introduced a single modification: a scalar weight β on the KL term of the objective,

$$\mathcal{L}_{\beta\text{-VAE}} = \mathcal{L}_{\text{recon}} + \beta D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})). \quad (2.6)$$

Setting $\beta > 1$ places additional pressure on the approximate posterior to match the factorised prior, which encourages individual latent dimensions to capture statistically independent factors of variation — a property known as *disentanglement*. The standard VAE is recovered at $\beta = 1$. Changing β offers a trade-off: a larger value improves the structure and interpretability of the latent space at the cost of reconstruction fidelity.

VQ-VAE Oord, Vinyals and Kavukcuoglu [21] replaced the continuous Gaussian latent with a *discrete* one. The encoder output is quantised to the nearest entry of a learned codebook $\{\mathbf{e}_1, \dots, \mathbf{e}_K\}$, so that each latent is represented by a discrete index rather than a continuous vector. This sidesteps the *posterior collapse* that can affect continuous VAEs paired with expressive autoregressive decoders, in which the decoder learns to ignore the latent entirely. Because the nearest-neighbour lookup is non-differentiable, gradients are passed to the encoder with a straight-through estimator and the codebook is learned through dedicated loss terms. VQ-VAE underpins many modern generative systems, providing the discrete latent vocabulary over which an autoregressive prior is subsequently trained.

2.3.2.3 Applications

VAEs are used both as practical components in large generative systems and for interpretable representation learning; the latter is the more relevant to this thesis.

check this

GENERATIVE MODELLING Although a VAE can act as a standalone generator, in modern systems it more often serves as a *compression backbone* for other generative models: latent diffusion models run the diffusion process in a VAE’s compact latent space rather than in pixel space, as in Stable Diffusion [25], while discrete VQ-VAEs [21] map data to token sequences that an autoregressive model can generate. The smooth latent also serves as a search space: optimising over it has been used to search for new molecules with desired properties [10].

ANOMALY DETECTION Like standard AEs, VAEs can flag anomalies, but they replace the single reconstruction error with a probabilistic score: the reconstruction probability, or the ELBO, gives a principled measure of how well the model explains an input [3]. This is the probabilistic counterpart of the approach described in section 2.3.1.1.

REPRESENTATION LEARNING VAEs are also a standard tool for unsupervised representation learning. The β -VAE (section 2.3.2.2) is the canonical example: up-weighting the KL term pressures the model towards *disentangled* latents whose individual dimensions capture independent, semantically meaningful factors of variation [13], yielding structured low-dimensional features for downstream use.

INTERPRETABILITY The structured latent that makes VAEs good representation learners also makes them attractive for explainability. Their smoothness supports *counterfactual explanation* — perturbing the latent yields plausible, on-manifold variants of an input — and a substantial line of work uses VAEs for *causal representation learning*, aligning latent factors with underlying causal variables [12, 16].

VAEs have also been paired with concept-based models such as self-explaining networks [1]. Closest to this thesis, variational ideas have been brought to sparse dictionary learning — variational sparse coding [30] and variational Sparse AutoEncoders [4] — seeking to combine the interpretability of sparse codes with the structured latent of a VAE, the same combination developed in section 3.3.

2.3.3 Sparse AutoEncoders

Sparse AutoEncoders (SAEs) are one of the main variants of AutoEncoders. Just like AEs, they learn a dictionary of feature directions from the input data, but they typically are overcomplete ($m \gg n$) and their latent representation is forced to be sparse using some kind of sparsity mechanism. Given input $\mathbf{x} \in \mathbb{R}^n$, the SAE encodes it as:

$$\mathbf{z} = \text{ReLU}(\mathbf{W}_e(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_e), \quad (2.7)$$

where $\mathbf{W}_e \in \mathbb{R}^{m \times n}$ is the encoder weight matrix, $\mathbf{b}_e \in \mathbb{R}^m$ is the encoder bias, and $\mathbf{b}_{\text{pre}} \in \mathbb{R}^n$ is a pre-encoder bias subtracted from the input before encoding. This learned offset allows the encoder to operate on roughly mean-centered activations, improving training stability [6]. The decoder then reconstructs the input as:

$$\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{z} + \mathbf{b}_{\text{pre}}, \quad (2.8)$$

where $\mathbf{W}_d \in \mathbb{R}^{n \times m}$ is the decoder weight matrix. Each column $\mathbf{d}_i$ of $\mathbf{W}_d$ is a dictionary atom, representing a candidate feature direction in activation space.

When used in the context of Mechanistic Interpretability (MechInt), the overcompleteness and the sparsity constraint are motivated by the *superposition hypothesis*.

2.3.3.1 The Superposition Hypothesis

A central challenge in MechInt is that individual neurons in neural networks tend to be *polysemantic*: they activate in response to multiple, seemingly unrelated concepts [8, 19]. An example of this can be seen in fig. 2.3. This runs contrary to the intuition that an interpretable model should have neurons with clean, well-defined roles.

Elhage et al. [8] proposed the *superposition hypothesis* to explain this phenomenon. The core idea is that a network may represent more features than it has dimensions, by exploiting the fact that natural inputs are sparse: at any given time, only a small fraction of all possible features are relevant. Under this condition, a network can pack many near-orthogonal feature directions into a lower-dimensional space with limited interference; polysemanticity ceases to be a failure of the model, and instead becomes a rational compression strategy under capacity constraints.

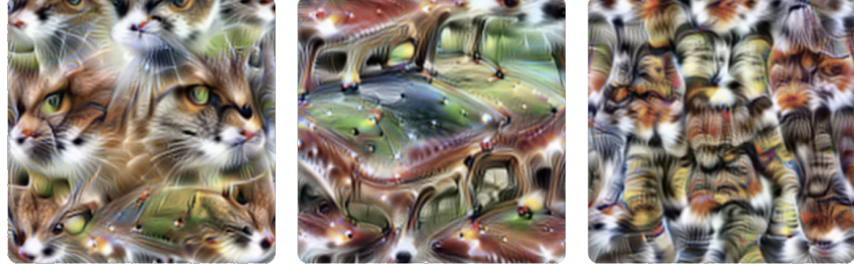


Figure 2.3: Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs [19].

This has a direct implication for interpretability: the natural basis of a model’s activations is not the neuron basis, but a larger, overcomplete set of feature directions embedded within it. Recovering this basis requires moving beyond individual neuron analysis, towards methods that can identify sparse, distributed structure. This is precisely the goal of *sparse dictionary learning* [20], a classical technique from computational neuroscience that **SAEs** adapt to the setting of modern language models.

2.3.3.2 Architectures

There are many ways to enforce sparsity, each with its pros and cons. Here is a selection of methods that range from the initial **SAE** idea to more modern architectures that try to improve on it.

VANILLA ℓ_1 This is the original architecture proposed by Huben et al. [14]. It uses an ℓ_1 ¹ penalty on the latent code to encourage most of $\mathbf{z}$ ’s entries to be exactly zero, so that any given activation is reconstructed from a small number of active features. The loss used to train this model is the following:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \|\mathbf{z}\|_1. \quad (2.9)$$

The scalar λ controls the sparsity–reconstruction trade-off and must be tuned carefully: too large, and reconstruction degrades; too small, and the learned features lose monosemanticity. A known side effect of ℓ_1 regularisation is *shrinkage* [29, 33]: beyond driving inactive features to zero, the penalty also biases active feature magnitudes below their reconstruction-optimal values, causing the model to systematically underestimate the true activation of each feature [22, 33]. Constraining the decoder columns to unit ℓ_2 norm after each gradient step partially mitigates this, by preventing the model from reducing the ℓ_1 penalty through the trivial reparameterisation of scaling down encoder outputs and compensating with larger decoder norms [22]. There is, however, a residual gradient-level bias that is structurally entangled with the ℓ_1 penalty and cannot be resolved with this trick.

¹ The ℓ_p norm is defined as $\ell_p(\mathbf{x}) = \|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$, $p \geq 1$.

TOPK SAES Gao et al. [9] proposed replacing the ℓ_1 penalty with a hard sparsity constraint, implemented via a TopK activation function that retains only the k largest pre-activations and zeros the rest. This directly controls the number of active features per forward pass, eliminating the need to tune λ and removing the shrinkage bias entirely. The full training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \alpha \mathcal{L}_{\text{aux}}, \quad (2.10)$$

where the auxiliary loss $\mathcal{L}_{\text{aux}}$ addresses the problem of dead latents, i. e., features that cease to activate entirely during training. Latents are flagged as dead if they have not activated for a predetermined number of tokens. Given the reconstruction residual $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$, the top- k_{aux} dead latents are selected and used to reconstruct $\mathbf{e}$; the resulting reconstruction error is $\mathcal{L}_{\text{aux}}$. This gives dead latents a gradient signal proportional to how much of the residual they could explain, encouraging them to activate rather than remain permanently dormant. Gao et al. [9] demonstrate that TopK SAES achieve better reconstruction at equivalent sparsity levels compared to ℓ_1 baselines, making them a strong and widely used alternative.

GATED SAES Rajamanoharan et al. [22] introduced an architecture that separates the decision of whether a feature is active from the decision of how strongly it activates. A dedicated gating pathway produces a binary mask via a Heaviside step function (approximated during training with a straight-through estimator) which is applied element-wise to the encoder output. This architectural separation allows the model to learn feature presence and magnitude more independently, addressing shrinkage from a structural rather than a loss-based perspective.

JUMPRELU SAES Rajamanoharan et al. [23] proposed replacing the standard ReLU with a JumpReLU activation, which is zero below a learned per-feature threshold θ_i and equal to the pre-activation above it:

$$\text{JumpReLU}_{\theta_i}(z_i) = z_i \cdot \mathbb{1}[z_i > \theta_i]. \quad (2.11)$$

The key advantage of this parameterisation is that it provides a dedicated, per-feature learnable threshold, which allows the ℓ_0 norm² of the feature activations — $\sum_i H(\pi_i - \theta_i)$, where π_i are the encoder pre-activations and $H(x) = \mathbb{1}[x > 0]$ is the Heaviside step function — to be trained directly via a straight-through estimator applied to the threshold parameter θ_i . Because each threshold can be positioned near the actual boundary between active and inactive pre-activations for its feature, the gradient signal is stable and controllable, making ℓ_0 training practical in a way that would not be possible with a fixed

² The ℓ_0 norm is a pseudo-norm defined as the number of non-zero vector components.

threshold. Training with ℓ_0 rather than ℓ_1 eliminates shrinkage entirely, since ℓ_0 penalises only the number of active features and not their intensity. JumpReLU SAEs achieve performance competitive with TopK SAEs.

2.3.3.3 Applications

Unlike general-purpose AEs and VAEs, which are applied across a range of domains, SAEs find their primary application in interpreting the internal activations of neural networks, and MechInt remains their most prominent application, making it the focus of this section. SAEs can be trained on activations from any layer or component of a transformer; common targets include the residual stream at various depths, MLP layer outputs, and attention head outputs [6, 14]. The resulting dictionary then supports the applications described below.

Make sure to mention what the residual stream, Multi-Layer Perceptron (MLP), and attention heads are in a chapter about LLMs

FEATURE DISCOVERY The primary use of SAEs is *feature discovery*: training a SAE on a model’s internal activations and examining the resulting dictionary atoms to identify human-interpretable concepts. The underlying hypothesis is that each atom, active for only a semantically coherent set of inputs, corresponds to a monosemantic feature of the model’s computation — a clean unit of meaning that the model had obscured in the neuron basis. In practice these features often correspond to recognisable concepts: Bricken et al. [6] trained a SAE on the MLP of a one-layer transformer and found features responsive to specific tokens, syntactic roles, and semantic categories, while at production scale Templeton et al. [28] report features for famous people, countries, emotions, or even biases (like racism, sexism, hatred) and sarcastic praise. These results provide strong empirical support for the superposition hypothesis and suggest that SAEs can serve as a practical lens through which to study what information LLMs represent.

STEERING The features recovered by a SAE are not merely descriptive: because each corresponds to a direction in activation space, its activation can be clamped to an arbitrary value during a forward pass to causally *steer* the model’s behaviour. Templeton et al. [28] demonstrated this by amplifying a single feature, causing the model to compulsively reference the Golden Gate Bridge regardless of the input. Such interventions show that the discovered features are causally implicated in the model’s computation rather than merely correlated with its inputs, and open a path to controllable generation and targeted safety interventions.

CIRCUIT ANALYSIS Beyond cataloguing individual features, SAEs can serve as the building blocks for reconstructing the *circuits* a model uses to perform a computation, expressing the interactions between

features across layers as an interpretable causal graph [18]. More recent work favours *transcoders* — a variant trained to approximate the input–output behaviour of an MLP sublayer with a sparse hidden representation, rather than to reconstruct a single activation — which should disentangle the model’s computation more cleanly and yield sparser, more faithful circuits [2, 7].

2.3.3.4 Evaluation

Assessing the quality of learned features is non-trivial, as interpretability is inherently subjective. Several proxy metrics are used in practice. *Reconstruction quality* is typically measured by the downstream Cross-Entropy loss recovered when replacing original activations with SAE reconstructions [9]. *Sparsity* is quantified via the average ℓ_0 norm of the latent codes, i.e., the mean number of active features per token. *Automated interpretability* pipelines use a second language model to generate and score natural-language descriptions of individual features [5], providing a scalable if imperfect proxy for human judgement.

Could this be made clearer?

CONCLUSIONS

The inherent tension between reconstruction quality, sparsity, and interpretability is a recurring theme in the SAE literature, and motivates both the architectural variants described in section 2.3.3.2 and the alternative approach developed in this thesis and described in section 3.3.

Rough seed — expand into a proper chapter conclusion.

METHODS

3.1 DEFINITIONS/NOTATION

3.2 AIM OF THE WORK

3.3 MODEL DEVELOPMENT

3.3.1 *Probabilistic Formulation*

3.3.2 *Architectures*

CONCLUSIONS

EXPERIMENTS

4.1 MODEL TRAINING

4.2 EVALUATION METRICS

4.2.1 *MSE*

4.2.2 ℓ_0

4.2.3 *Perplexity*

4.3 RESULTS

CONCLUSIONS

CONCLUSIONS

5.1 CONTRIBUTIONS

5.2 FUTURE WORK

BIBLIOGRAPHY

- [1] David Alvarez-Melis and Tommi S. Jaakkola. *Towards Robust Interpretability with Self-Explaining Neural Networks*. Dec. 2018. DOI: [10.48550/arXiv.1806.07538](#). arXiv: [1806.07538 \[cs\]](#). (Visited on 16/10/2025).
- [2] Emmanuel Ameisen et al. 'Circuit Tracing: Revealing Computational Graphs in Language Models'. In: *Transformer Circuits Thread* (Mar. 2025). (Visited on 10/06/2026).
- [3] Jinwon An and Sungzoon Cho. *Variational Autoencoder Based Anomaly Detection Using Reconstruction Probability*. Dec. 2015.
- [4] Zachary Baker and Yuxiao Li. *Analysis of Variational Sparse Autoencoders*. Oct. 2025. DOI: [10.48550/arXiv.2509.22994](#). arXiv: [2509.22994 \[cs\]](#). (Visited on 20/10/2025).
- [5] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu and William Saunders. 'Language Models Can Explain Neurons in Language Models'. In: *OpenAI* (May 2023). (Visited on 29/05/2026).
- [6] Trenton Bricken et al. 'Towards Monosemanticity: Decomposing Language Models With Dictionary Learning'. In: *Transformer Circuits Thread* (Oct. 2023). (Visited on 28/05/2026).
- [7] Jacob Dunefsky, Philippe Chlenski and Neel Nanda. *Transcoders Find Interpretable LLM Feature Circuits*. Nov. 2024. DOI: [10.48550/arXiv.2406.11944](#). arXiv: [2406.11944 \[cs\]](#). (Visited on 24/10/2025).
- [8] Nelson Elhage et al. 'Toy Models of Superposition'. In: *Transformer Circuits Thread* (Sept. 2022). (Visited on 29/05/2026).
- [9] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike and Jeffrey Wu. 'Scaling and Evaluating Sparse Autoencoders'. In: *The Thirteenth International Conference on Learning Representations*. Oct. 2024. (Visited on 14/05/2026).
- [10] Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams and Alán Aspuru-Guzik. 'Automatic Chemical Design Using a Data-Driven Continuous Representation of Molecules'. In: *ACS Central Science* 4.2 (Feb. 2018), pp. 268–276. ISSN: 2374-7943. DOI: [10.1021/acscentsci.7b00572](#). (Visited on 10/06/2026).

- [11] Ian Goodfellow, Aaron Courville and Yoshua Bengio. *Deep Learning*. Adaptive Computation and Machine Learning. Cambridge, Massachusetts: The MIT Press, 2016. ISBN: 978-0-262-03561-3 978-0-262-33737-3.
- [12] Yash Goyal, Amir Feder, Uri Shalit and Been Kim. *Explaining Classifiers with Causal Concept Effect (CaCE)*. Feb. 2020. DOI: [10.48550/arXiv.1907.07165](#). arXiv: [1907.07165 \[cs.LG\]](#). (Visited on 01/06/2026).
- [13] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed and Alexander Lerchner. ‘Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework’. In: *International Conference on Learning Representations*. Feb. 2017. (Visited on 10/06/2026).
- [14] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart and Lee Sharkey. ‘Sparse Autoencoders Find Highly Interpretable Features in Language Models’. In: *The Twelfth International Conference on Learning Representations*. Oct. 2023. (Visited on 28/05/2026).
- [15] Diederik P. Kingma and Max Welling. *Auto-Encoding Variational Bayes*. Dec. 2022. DOI: [10.48550/arXiv.1312.6114](#). arXiv: [1312.6114 \[stat\]](#). (Visited on 07/11/2025).
- [16] Aneesh Komanduri, Yongkai Wu, Wen Huang, Feng Chen and Xintao Wu. ‘SCM-VAE: Learning Identifiable Causal Representations via Structural Knowledge’. In: *2022 IEEE International Conference on Big Data (Big Data)*. Dec. 2022, pp. 1014–1023. DOI: [10.1109/BigData55660.2022.10021114](#). (Visited on 21/04/2026).
- [17] Alex Krizhevsky and Geoffrey E. Hinton. ‘Using Very Deep Autoencoders for Content-Based Image Retrieval’. In: *19th European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning* (Apr. 2011).
- [18] Samuel Marks, Can Rager, Eric Michaud, Yonatan Belinkov, David Bau and Aaron Mueller. ‘Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models’. In: *International Conference on Learning Representations 2025* (May 2025), pp. 23888–23923. (Visited on 10/06/2026).
- [19] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov and Shan Carter. ‘Zoom In: An Introduction to Circuits’. In: *Distill* 5.3 (Mar. 2020), e00024.001. ISSN: 2476-0757. DOI: [10.23915/distill.00024.001](#). (Visited on 04/05/2026).
- [20] Bruno A. Olshausen and David J. Field. ‘Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by V1?’ In: *Vision Research* 37.23 (Dec. 1997), pp. 3311–3325. ISSN: 0042-6989. DOI: [10.1016/S0042-6989\(97\)00169-7](#). (Visited on 28/05/2026).

- [21] Aaron van den Oord, Oriol Vinyals and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. May 2018. DOI: [10.48550/arXiv.1711.00937](https://doi.org/10.48550/arXiv.1711.00937). arXiv: [1711.00937](https://arxiv.org/abs/1711.00937) [cs]. (Visited on 23/02/2026).
- [22] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah and Neel Nanda. ‘Improving Sparse Decomposition of Language Model Activations with Gated Sparse Autoencoders’. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 775–818. DOI: [10.52202/079017-0024](https://doi.org/10.52202/079017-0024). (Visited on 05/06/2026).
- [23] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár and Neel Nanda. *Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders*. Aug. 2024. DOI: [10.48550/arXiv.2407.14435](https://doi.org/10.48550/arXiv.2407.14435). arXiv: [2407.14435](https://arxiv.org/abs/2407.14435) [cs]. (Visited on 23/02/2026).
- [24] Danilo Jimenez Rezende, Shakir Mohamed and Daan Wierstra. ‘Stochastic Backpropagation and Approximate Inference in Deep Generative Models’. In: *Proceedings of the 31st International Conference on Machine Learning*. PMLR, June 2014, pp. 1278–1286. (Visited on 10/06/2026).
- [25] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Bjorn Ommer. ‘High-Resolution Image Synthesis with Latent Diffusion Models’. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, June 2022, pp. 10674–10685. ISBN: 978-1-6654-6946-3. DOI: [10.1109/CVPR52688.2022.01042](https://doi.org/10.1109/CVPR52688.2022.01042). (Visited on 10/06/2026).
- [26] Mayu Sakurada and Takehisa Yairi. ‘Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction’. In: *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*. MLSDA’14. New York, NY, USA: Association for Computing Machinery, Dec. 2014, pp. 4–11. ISBN: 978-1-4503-3159-3. DOI: [10.1145/2689746.2689747](https://doi.org/10.1145/2689746.2689747). (Visited on 05/06/2026).
- [27] Irhum Shafkat. *Intuitively Understanding Variational Autoencoders*. Feb. 2018. (Visited on 14/05/2026).
- [28] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html#appendix-citation>. (Visited on 20/10/2025).
- [29] Robert Tibshirani. ‘Regression Shrinkage and Selection Via the Lasso’. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1 (Jan. 1996), pp. 267–288. ISSN: 0035-9246. DOI: [10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x). (Visited on 29/05/2026).

- [30] Francesco Tonolini, Bjørn Sand Jensen and Roderick Murray-Smith. ‘Variational Sparse Coding’. In: *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*. PMLR, Aug. 2020, pp. 690–700. (Visited on 26/03/2026).
- [31] Pascal Vincent, Hugo Larochelle, Yoshua Bengio and Pierre-Antoine Manzagol. ‘Extracting and Composing Robust Features with Denoising Autoencoders’. In: *Proceedings of the 25th International Conference on Machine Learning - ICML ’08*. Helsinki, Finland: ACM Press, 2008, pp. 1096–1103. ISBN: 978-1-60558-205-4. DOI: [10.1145/1390156.1390294](https://doi.org/10.1145/1390156.1390294). (Visited on 05/06/2026).
- [32] Lilian Weng. *From Autoencoder to Beta-VAE*. Aug. 2018. (Visited on 05/06/2026).
- [33] Benjamin Wright and Lee Sharkey. ‘Addressing Feature Suppression in SAEs — AI Alignment Forum’. In: (Feb. 2024). (Visited on 29/05/2026).

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst’s seminal book on typography “*The Elements of Typographic Style*”. `classicthesis` is available for both $\text{\LaTeX}$ and $\text{\LyX}$:

<https://bitbucket.org/amiede/classicthesis/>

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

<http://postcards.miede.de/>

Thank you very much for your feedback and contribution.